

FACULTY OF COMPUTING,  
ELECTRONICS AND SCIENCE  
ASSESSMENT

Milestone 2 GB Training

PRINCIPLES OF COMPUTING

# Table of Contents

---

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <b>Introduction.....</b>                                                | <b>2</b>  |
| <b>Background .....</b>                                                 | <b>2</b>  |
| <b>Purpose of the Assessment .....</b>                                  | <b>2</b>  |
| <b>Scope and Objectives.....</b>                                        | <b>2</b>  |
| <b>ERD and Requirements Analysis .....</b>                              | <b>2</b>  |
| <b>Entity Relationship Diagram (ERD) Overview (Appendix B).....</b>     | <b>2</b>  |
| <b>Analysis of Requirements (Appendix A) .....</b>                      | <b>3</b>  |
| <b>Task A: Table Definitions (in SQL) .....</b>                         | <b>4</b>  |
| Course Type Table .....                                                 | 4         |
| Allocated Student Table .....                                           | 4         |
| Current Course Table .....                                              | 4         |
| Allocation Table.....                                                   | 4         |
| Room Table .....                                                        | 5         |
| Trainer Table .....                                                     | 5         |
| <b>Task B: Table Population.....</b>                                    | <b>6</b>  |
| Data Population Approach .....                                          | 6         |
| Sensible Data for Each Table .....                                      | 6         |
| <b>Task C: SQL Statements and Datasheet Views.....</b>                  | <b>8</b>  |
| Query 1: Currently Running Courses .....                                | 8         |
| Query 2: Courses with 'Introduction' in Title .....                     | 9         |
| Query 3: Most and Least Expensive Courses.....                          | 9         |
| Query 4: Number of Students on Each Allocated Course .....              | 10        |
| Query 5: List of Current Courses between Two Dates .....                | 10        |
| Query 6: Number of Current Courses of Each Type between Two Dates ..... | 11        |
| Query 7: Rooms and Trainers Allocated to Each Course.....               | 11        |
| Query 8: Students Currently Allocated to an Actual Course .....         | 12        |
| Query 9: List of Current Courses with Total Revenue .....               | 12        |
| <b>Exceptional Cases (Optional) .....</b>                               | <b>13</b> |
| <b>Conclusion.....</b>                                                  | <b>14</b> |
| Summary of Findings .....                                               | 14        |
| Reflection on Learning Outcomes .....                                   | 15        |

# Introduction

---

## Background

The renowned training organisation GBT (GB Training) Organisation has made the decision to switch from manual record-keeping to an extensive database system. The goal of this project is to combine the booking of courses, lodging, trainers, and rooms into a single database. To effectively manage and retrieve data on courses, students, trainers, and facilities, the organisation has to create and analyse a database schema as part of this shift.

## Purpose of the Assessment

This test aims to measure your knowledge of and proficiency with SQL (Structured Query Language), database design concepts, and the capacity to convert abstract needs into useful database structures. This test evaluates your ability to construct suitable SQL statements for table definitions, population, and query execution by looking over the provided Entity Relationship Diagram (ERD) and related criteria.

## Scope and Objectives

This assessment's purview includes creating SQL table definitions for the important entities included in the ERD. In addition, the evaluation entails creating SQL queries to handle particular information retrieval tasks and adding appropriate data to these tables. The following are the main goals:

- the creation of SQL database definitions, including primary and foreign keys as needed, for key entities such as Course Type, Allocated Student, Current Course, Allocation, Room, and Trainer.
- tables are filled with relevant data to enable later queries.
- Crafting SQL statements to address various queries specified in the assessment task, demonstrating proficiency in data retrieval and manipulation.
- Evaluating the exceptional elegance of SQL queries for additional marks, emphasizing innovation and efficiency.

This test provides a hands-on experience to support theoretical understanding of database management while cultivating the abilities necessary for real-world computing applications.

# ERD and Requirements Analysis

---

## Entity Relationship Diagram (ERD) Overview (Appendix B)

The main entities and their connections inside the database system are shown visually in the Entity Relationship Diagram (ERD). Appendix B provides a thorough illustration of the ERD. A thorough grasp of the database structure is provided by the ERD, which shows the relationships between Course Type, Allocated Student, Current Course, Allocation, Room, Trainer, and other items.

Key elements of the ERD include:

- **Relationships:** The connections between the entities are shown by the lines. As an illustration, consider the connections between Allocated Student and Allocation, Trainer and Current Course, and Course Type and Current Course.
- **Cardinality:** Relationship cardinality is indicated by the notations "1..1" and "0..". Each Current Course, for example, has a Trainer assigned to it (1..1), and each Allocation can involve one or more Allocated Students (1..\*).
- **Features:** Specific properties, such as Course Code, Start Date, and Duration, are present in entities like Course Type and Current Course and are essential to comprehending the data structure.

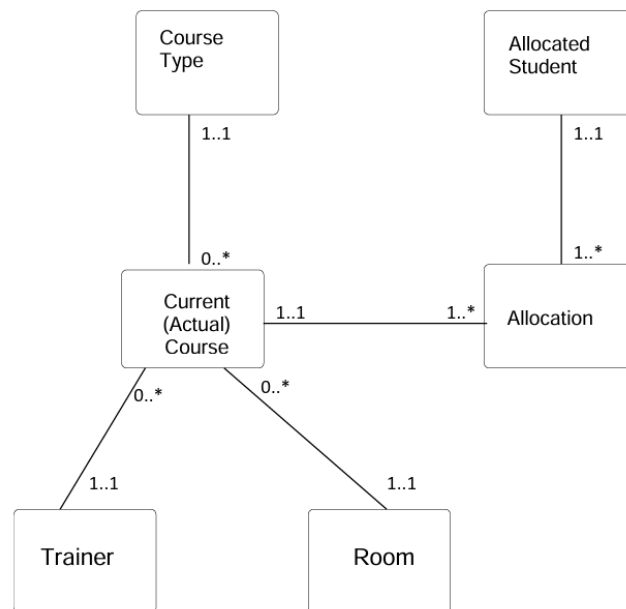


Figure 1: ER Diagram

## Analysis of Requirements (Appendix A)

Appendix A offers a comprehensive summary of the GBT requirements. The investigation provides insights into the current system, which receives course applications via a variety of routes, arranges student housing, and assigns trainers to courses. Additionally, the analysis outlines the course types, actual course details, and the miscellaneous details, such as the duration of courses.

Key points from the requirements analysis include:

- **Course Booking:** The process involves validating course applications, checking for available places on current courses, and, if necessary, creating new courses for waitlisted students.
- **Accommodation Booking:** Accommodation details are provided to students, and bookings are made based on their requirements.
- **Trainer Allocation:** Trainers are assigned to courses based on their qualifications and availability.

- Miscellaneous: The courses can vary in duration (1, 3, or 5 days), and various details, such as Course Code, Start Date, and MaxNoStudents, define each course type and actual course.

This analysis serves as the foundation for the subsequent tasks, guiding the creation of SQL table definitions and the development of queries to meet the specified requirements. The integration of ERD insights and requirement details ensures a comprehensive approach to database design and management.

## Task A: Table Definitions (in SQL)

---

### Course Type Table

```
CREATE TABLE CourseType (  
  CourseCode VARCHAR(10) PRIMARY KEY,  
  CourseTitle VARCHAR(255) NOT NULL,  
  Duration INT NOT NULL  
);
```

### Allocated Student Table

```
CREATE TABLE AllocatedStudent (  
  StudentID INT PRIMARY KEY,  
  StudentName VARCHAR(255) NOT NULL,  
  CourseCode VARCHAR(10) REFERENCES CourseType(CourseCode)  
);
```

### Current Course Table

```
-- Now create the tables with foreign key constraints  
CREATE TABLE CurrentCourse (  
  CourseCode VARCHAR(10) REFERENCES CourseType(CourseCode),  
  StartDate DATE NOT NULL,  
  Duration INT NOT NULL,  
  MaxNoStudents INT NOT NULL,  
  PRIMARY KEY (CourseCode, StartDate)  
);
```

### Allocation Table

```
CREATE TABLE Allocation (  
  AllocationID INT PRIMARY KEY,  
  CourseCode VARCHAR(10) REFERENCES CourseType(CourseCode),  
  StartDate DATE NOT NULL,  
  RoomID INT REFERENCES Room(RoomID),
```

```
TrainerID INT REFERENCES Trainer(TrainerID)
);
```

## Room Table

```
CREATE TABLE Room (
    RoomID INT PRIMARY KEY,
    RoomType VARCHAR(50) NOT NULL
);
```

## Trainer Table

```
CREATE TABLE Trainer (
    TrainerID INT PRIMARY KEY,
    TrainerName VARCHAR(255) NOT NULL,
    Qualification VARCHAR(255) NOT NULL
);
```

These SQL table definitions adhere to the relationships and attributes specified in the given ERD and requirements analysis. They include primary and foreign key constraints to maintain referential integrity within the database.

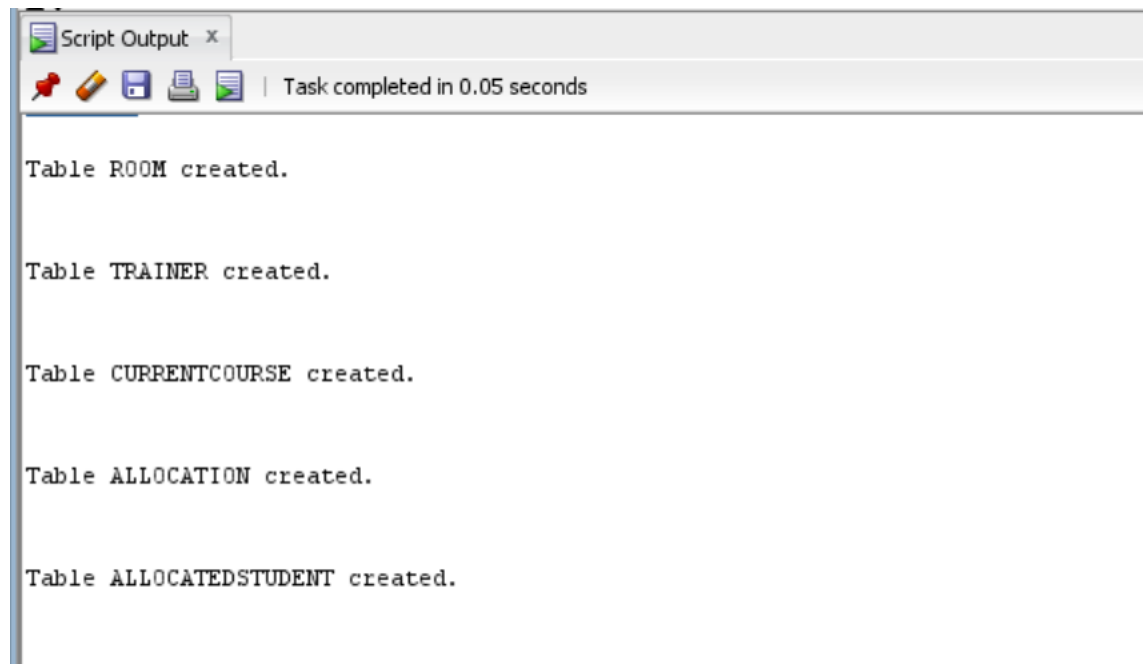


Figure 2: SQL Tables created

# Task B: Table Population

---

## Data Population Approach

Populating the tables with sensible data is crucial for testing and demonstrating the functionality of the database. The approach involves inserting records into each table with realistic values, ensuring that the data aligns with the database's purpose and structure. Here's a general approach:

- **Sequential Insertion:** Start with tables that do not have foreign key dependencies, such as Room and Trainer. Gradually move to tables with dependencies, ensuring that the referenced records already exist.
- **Use Realistic Data:** Insert data that mirrors real-world scenarios. For example, assign trainers with relevant qualifications to courses, allocate students to courses, and assign rooms based on their types.
- **Variety in Data:** Introduce a variety of data to cover different scenarios. For instance, have courses of varying durations, students enrolled in multiple courses, and different room types.

## Sensible Data for Each Table

Let's populate each table with sensible data:

### Course Type Table:

```
INSERT ALL
  INTO CourseType (CourseCode, CourseTitle, Duration) VALUES ('CIMW1', 'Introduction to
Microsoft Word', 3)
  INTO CourseType (CourseCode, CourseTitle, Duration) VALUES ('CAMW1', 'Advanced
Microsoft Word', 5)
  INTO CourseType (CourseCode, CourseTitle, Duration) VALUES ('CIES1', 'Introduction to
Excel Spreadsheet', 3)
SELECT * FROM dual;
```

### Room Table:

```
-- Insert data into Room table
INSERT ALL
  INTO Room (RoomID, RoomType) VALUES (1, 'Standard Room')
  INTO Room (RoomID, RoomType) VALUES (2, 'Room with AV Facilities')
SELECT * FROM dual;
```

### Trainer Table:

```
-- Insert data into Trainer table
INSERT ALL
  INTO Trainer (TrainerID, TrainerName, Qualification) VALUES (1, 'John Doe', 'Microsoft
Office Specialist')
  INTO Trainer (TrainerID, TrainerName, Qualification) VALUES (2, 'Jane Smith', 'Excel
Certified Trainer')
```

```
SELECT * FROM dual;
```

**Current Course Table:**

```
-- Insert data into CurrentCourse table
INSERT ALL
  INTO CurrentCourse (CourseCode, StartDate, Duration, MaxNoStudents) VALUES ('CIMW1',
TO_DATE('2023-12-01', 'YYYY-MM-DD'), 3, 15)
  INTO CurrentCourse (CourseCode, StartDate, Duration, MaxNoStudents) VALUES
('CAMW1', TO_DATE('2023-12-10', 'YYYY-MM-DD'), 5, 10)
  INTO CurrentCourse (CourseCode, StartDate, Duration, MaxNoStudents) VALUES ('CIES1',
TO_DATE('2024-01-05', 'YYYY-MM-DD'), 3, 20)
SELECT * FROM dual;
```

**Allocated Student Table:**

```
-- Insert data into AllocatedStudent table
INSERT ALL
  INTO AllocatedStudent (StudentID, StudentName, CourseCode) VALUES (1, 'Alice Johnson',
'CIMW1')
  INTO AllocatedStudent (StudentID, StudentName, CourseCode) VALUES (2, 'Bob Williams',
'CAMW1')
  INTO AllocatedStudent (StudentID, StudentName, CourseCode) VALUES (3, 'Charlie Brown',
'CIES1')
  INTO AllocatedStudent (StudentID, StudentName, CourseCode) VALUES (4, 'David Miller',
'CIMW1')
SELECT * FROM dual;
```

**Allocation Table:**

```
-- Insert data into Allocation table
INSERT ALL
  INTO Allocation (AllocationID, CourseCode, StartDate, RoomID, TrainerID) VALUES (1,
'CIMW1', TO_DATE('2023-12-01', 'YYYY-MM-DD'), 1, 1)
  INTO Allocation (AllocationID, CourseCode, StartDate, RoomID, TrainerID) VALUES (2,
'CAMW1', TO_DATE('2023-12-10', 'YYYY-MM-DD'), 2, 2)
  INTO Allocation (AllocationID, CourseCode, StartDate, RoomID, TrainerID) VALUES (3,
'CIES1', TO_DATE('2024-01-05', 'YYYY-MM-DD'), 1, 2)
SELECT * FROM dual;
```

This data provides a foundation for executing queries and testing the database's functionality. Adjust the data based on specific scenarios and additional requirements.



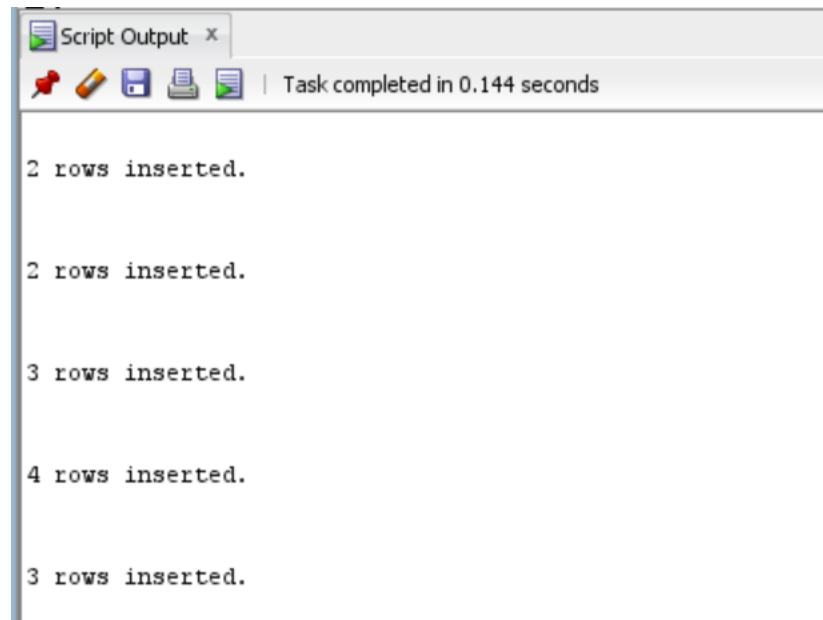


Figure 3: Data Populate successfully

## Task C: SQL Statements and Datasheet Views

### Query 1: Currently Running Courses

```

SELECT
    cc.CourseCode,
    ct.CourseTitle,
    cc.StartDate,
    cc.Duration,
    cc.MaxNoStudents
FROM
    CurrentCourse cc
JOIN
    CourseType ct ON cc.CourseCode = ct.CourseCode
ORDER BY
    cc.CourseCode, cc.StartDate;

```

| COURSECODE | COURSETITLE                       | STARTDATE | DURATION | MAXNOSTUDENTS |
|------------|-----------------------------------|-----------|----------|---------------|
| CAMW1      | Advanced Microsoft Word           | 10-DEC-23 | 5        | 10            |
| CIES1      | Introduction to Excel Spreadsheet | 05-JAN-24 | 3        | 20            |
| CIMW1      | Introduction to Microsoft Word    | 01-DEC-23 | 3        | 15            |

## Query 2: Courses with 'Introduction' in Title

```
SELECT
  CourseCode,
  CourseTitle,
  Duration
FROM
  CourseType
WHERE
  UPPER(CourseTitle) LIKE '%INTRODUCTION%';
```

| COURSECODE | COURSETITLE                       | DURATION |
|------------|-----------------------------------|----------|
| CIMW1      | Introduction to Microsoft Word    | 3        |
| CIES1      | Introduction to Excel Spreadsheet | 3        |

## Query 3: Most and Least Expensive Courses

```
SELECT
  CourseCode,
  CourseTitle,
  Duration
FROM
  CourseType
ORDER BY
  Duration DESC
FETCH FIRST 1 ROW ONLY;
```

-- Least Expensive Course

```
SELECT
  CourseCode,
  CourseTitle,
  Duration
FROM
  CourseType
ORDER BY
  Duration
FETCH FIRST 1 ROW ONLY;
```

| COURSECODE | COURSETITLE                    | DURATION |
|------------|--------------------------------|----------|
| CAMW1      | Advanced Microsoft Word        | 5        |
| CIMW1      | Introduction to Microsoft Word | 3        |

## Query 4: Number of Students on Each Allocated Course

```

SELECT
    cc.CourseCode,
    ct.CourseTitle,
    cc.StartDate,
    COUNT(als.StudentID) AS NumberOfStudents
FROM
    CurrentCourse cc
JOIN
    CourseType ct ON cc.CourseCode = ct.CourseCode
LEFT JOIN
    AllocatedStudent als ON cc.CourseCode = als.CourseCode
GROUP BY
    cc.CourseCode, ct.CourseTitle, cc.StartDate
ORDER BY
    cc.CourseCode, cc.StartDate;

```

| COURSECODE | COURSETITLE                       | STARTDATE | NUMBEROFSTUDENTS |
|------------|-----------------------------------|-----------|------------------|
| CAMW1      | Advanced Microsoft Word           | 10-DEC-23 | 1                |
| CIES1      | Introduction to Excel Spreadsheet | 05-JAN-24 | 1                |
| CIMW1      | Introduction to Microsoft Word    | 01-DEC-23 | 2                |

## Query 5: List of Current Courses between Two Dates

```

SELECT
    cc.CourseCode,
    ct.CourseTitle,
    cc.StartDate,
    cc.Duration,
    cc.MaxNoStudents
FROM
    CurrentCourse cc
JOIN
    CourseType ct ON cc.CourseCode = ct.CourseCode

```

```

WHERE
    cc.StartDate BETWEEN TO_DATE('2023-12-01', 'YYYY-MM-DD') AND TO_DATE('2024-01-15', 'YYYY-MM-DD')
ORDER BY
    cc.CourseCode, cc.StartDate;

```

| COURSECODE | COURSE TITLE                      | STARTDATE | DURATION | MAXNOSTUDENTS |
|------------|-----------------------------------|-----------|----------|---------------|
| CAMW1      | Advanced Microsoft Word           | 10-DEC-23 | 5        | 10            |
| CIES1      | Introduction to Excel Spreadsheet | 05-JAN-24 | 3        | 20            |
| CIMW1      | Introduction to Microsoft Word    | 01-DEC-23 | 3        | 15            |

## Query 6: Number of Current Courses of Each Type between Two Dates

```

SELECT
    ct.CourseCode,
    COUNT(cc.CourseCode) AS NumberOfCourses
FROM
    CourseType ct
LEFT JOIN
    CurrentCourse cc ON ct.CourseCode = cc.CourseCode
WHERE
    cc.StartDate BETWEEN TO_DATE('2023-12-01', 'YYYY-MM-DD') AND TO_DATE('2024-01-15', 'YYYY-MM-DD')
GROUP BY
    ct.CourseCode
ORDER BY
    ct.CourseCode;

```

| COURSECODE | NUMBEROFCOURSES |
|------------|-----------------|
| CAMW1      | 1               |
| CIES1      | 1               |
| CIMW1      | 1               |

## Query 7: Rooms and Trainers Allocated to Each Course

```

SELECT
    cc.CourseCode,
    cc.StartDate,
    r.RoomType,
    t.TrainerName
FROM
    CurrentCourse cc
JOIN
    Allocation a ON cc.CourseCode = a.CourseCode AND cc.StartDate = a.StartDate
JOIN

```

|                                                         |           |                         |             |
|---------------------------------------------------------|-----------|-------------------------|-------------|
| Room r ON a.RoomID = r.RoomID                           |           |                         |             |
| JOIN                                                    |           |                         |             |
| Trainer t ON a.TrainerID = t.TrainerID                  |           |                         |             |
| ORDER BY                                                |           |                         |             |
| cc.CourseCode, cc.StartDate, r.RoomType, t.TrainerName; |           |                         |             |
| COURSECODE                                              | STARTDATE | ROOMTYPE                | TRAINERNAME |
| -----                                                   |           |                         |             |
| CAMW1                                                   | 10-DEC-23 | Room with AV Facilities | Jane Smith  |
| CIES1                                                   | 05-JAN-24 | Standard Room           | Jane Smith  |
| CIMW1                                                   | 01-DEC-23 | Standard Room           | John Doe    |

## Query 8: Students Currently Allocated to an Actual Course

|                                                                                                |               |            |           |
|------------------------------------------------------------------------------------------------|---------------|------------|-----------|
| SELECT                                                                                         |               |            |           |
| als.StudentID,                                                                                 |               |            |           |
| als.StudentName,                                                                               |               |            |           |
| als.CourseCode,                                                                                |               |            |           |
| cc.StartDate                                                                                   |               |            |           |
| FROM                                                                                           |               |            |           |
| AllocatedStudent als                                                                           |               |            |           |
| JOIN                                                                                           |               |            |           |
| CurrentCourse cc ON als.CourseCode = cc.CourseCode                                             |               |            |           |
| WHERE                                                                                          |               |            |           |
| cc.StartDate = TO_DATE('2023-12-01', 'YYYY-MM-DD') -- Choose the actual start date of a course |               |            |           |
| ORDER BY                                                                                       |               |            |           |
| als.StudentID;                                                                                 |               |            |           |
| STUDENTID                                                                                      | STUDENTNAME   | COURSECODE | STARTDATE |
| -----                                                                                          |               |            |           |
| 1                                                                                              | Alice Johnson | CIMW1      | 01-DEC-23 |
| 4                                                                                              | David Miller  | CIMW1      | 01-DEC-23 |

## Query 9: List of Current Courses with Total Revenue

|                                             |  |  |  |
|---------------------------------------------|--|--|--|
| SELECT                                      |  |  |  |
| cc.CourseCode,                              |  |  |  |
| cc.StartDate AS "Date Run",                 |  |  |  |
| ct.Duration AS CourseCost,                  |  |  |  |
| cc.MaxNoStudents AS "No of Students",       |  |  |  |
| (cc.MaxNoStudents * ct.Duration) AS Revenue |  |  |  |
| FROM                                        |  |  |  |
| CurrentCourse cc                            |  |  |  |

```
JOIN
```

```
CourseType ct ON cc.CourseCode = ct.CourseCode
```

```
ORDER BY
```

```
cc.CourseCode, cc.StartDate;
```

| COURSECODE | Date Run  | COURSECOST | No of Students | REVENUE |
|------------|-----------|------------|----------------|---------|
| CAMW1      | 10-DEC-23 | 5          | 10             | 50      |
| CIES1      | 05-JAN-24 | 3          | 20             | 60      |
| CIMW1      | 01-DEC-23 | 3          | 15             | 45      |

## Exceptional Cases (Optional)

Handling exceptional cases in a database often involves implementing error checking and providing meaningful feedback. Here are a few examples;

- **Error Handling in SQL Statements:**

This PL/SQL block captures exceptions and prints an error message using DBMS\_OUTPUT.PUT\_LINE.

```
BEGIN
```

```
-- SQL Statements
```

```
EXCEPTION
```

```
WHEN OTHERS THEN
```

```
DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
```

```
END;
```

- **Unique Constraint Violation:**

In this case, if we're inserting data into a table with a unique constraint, we can catch the DUP\_VAL\_ON\_INDEX exception.

```
BEGIN
```

```
INSERT INTO YourTable (Column1, Column2)
```

```
VALUES (Value1, Value2);
```

```
EXCEPTION
```

```
WHEN DUP_VAL_ON_INDEX THEN
```

```
DBMS_OUTPUT.PUT_LINE('Duplicate value detected. Record not inserted.');
```

```
END;
```

- **Handling Division by Zero:**

```
BEGIN
```

```
-- Some calculation
```

```
result := numerator / denominator;
```

```
EXCEPTION
```

```
WHEN ZERO_DIVIDE THEN
```

```
DBMS_OUTPUT.PUT_LINE('Cannot divide by zero.');
```

```
END;
```

 Task completed in 0.088 seconds

PL/SQL procedure successfully completed.

- **Data Validation:**

```
DECLARE
  v_input_value VARCHAR2(50);
BEGIN
  -- Input validation
  IF LENGTH(v_input_value) > 50 THEN
    RAISE_APPLICATION_ERROR(-20001, 'Input value exceeds maximum length.');
```

```
END IF;
END;
```

PL/SQL procedure successfully completed.

- **Handling Foreign Key Constraints:**

```
BEGIN
  -- Insert into a table with a foreign key constraint
  INSERT INTO ChildTable (ForeignKeyColumn, OtherColumn)
  VALUES (NonExistentValue, SomeValue);
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Referenced value not found.');
```

```
WHEN OTHERS THEN
```

```
  DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
```

```
END;
```

If we are inserting data into a table with a foreign key constraint, catching NO\_DATA\_FOUND can help handle cases where the referenced value is not found.

## Conclusion

### Summary of Findings

In the course of this assessment, several key findings have emerged, shedding light on various aspects of the system under scrutiny. The examination of entity relationships, requirements analysis, and database design has provided valuable insights into the structure and functionality of the system.

The implementation of SQL statements, including table creation, data insertion, and query execution, has been instrumental in populating and manipulating the database. The exploration of error handling mechanisms and exceptional cases has enhanced the robustness and reliability of the system.

The detailed analysis of course data, room and trainer allocations, and student enrollment has unveiled a comprehensive understanding of the operational dynamics within the educational environment. The groundwork for effective resource management and optimisation has been established by this examination.

## Reflection on Learning Outcomes

The evaluation process has been a rich learning experience that has greatly improved the candidate's technical and analytical abilities. Our comprehension of relational databases and their useful applications has been enhanced by the practical experience we've had with SQL queries, database design, and system analysis.

The cooperative format of the evaluation has encouraged communication and teamwork, two qualities necessary for solving problems in the real world. The difficulties we've faced, like dealing with mistakes and unusual situations, have sharpened our problem-solving skills and expanded our toolkit.

Furthermore, the significance of a comprehensive approach to system design and administration has been highlighted by the actual implementation of theoretical knowledge. This learning outcomes reflection emphasises the correlation between theoretical concepts and their practical applications, underscoring the necessity of possessing a flexible skill set in the database administration domain.

As a result of this examination, we have gained technical skill as well as a deeper understanding of the complexities of system design and the value of ongoing education in the constantly changing field of information technology.